

TECHNICAL REQUIREMENTS DOCUMENT

# The Engineering Contract

Architecture, tech stack, non-functional requirements, and the local vision-AI pipeline in detail.

# What's Inside

---

## 01 Architecture Overview

How the Rust core, React shell, and local AI runtime fit together.

---

## 02 Technology Stack

Every layer, its version target, and why it was chosen.

---

## 03 Non-Functional Requirements

Performance, reliability, security, and portability targets.

---

## 04 Local Vision AI Pipeline

The exact request/response contract with the Ollama runtime.

---

## 05 Data Flow & Security

How a photo becomes a database row, and what never leaves the device.

---

## 06 Error Handling & Resilience

What happens when the model, the daemon, or the disk misbehaves.

---

## 07 Cross-Platform Notes

Windows, macOS, and Linux-specific implementation details.

---

## 01 — SYSTEM DESIGN

# Architecture Overview

AuraFit AI is a three-layer local system: a React/TypeScript presentation layer, a Rust command layer running inside the Tauri v2 shell, and two local services it talks to on localhost — the Ollama inference daemon and an embedded SQLite file. No layer requires outbound internet access.

## Presentation Layer

React 18 + TypeScript, Tailwind CSS, shadcn/ui components, Recharts for visualization. Communicates with Rust exclusively via Tauri's typed `invoke()` commands — never direct file or network access.

## Core / Command Layer

Rust backend inside the Tauri process. Owns file I/O, image encoding, the Ollama REST client, SQLite access, and all business logic (BMR/TDEE/macro math).

## Local Services

Ollama server on `http://localhost:11434` for inference; a single-file SQLite database (`tauri-plugin-sql`) for persistence. Both run entirely on-device.

### TRUST BOUNDARY

The only network socket AuraFit AI opens is a loopback connection to `localhost:11434` for Ollama. This is enforced at the Tauri capability/CSP level — the webview has no `http://` or `https://` permission to any external host.

## 1.1 Component Diagram

1

### React Webview (Renderer)

Renders UI, holds local component state, calls `invoke("command_name", payload)`.

2

### Tauri IPC Bridge

Serializes/deserializes the call across the process boundary; enforces the allow-list of exposed commands.

3

### Rust Command Handlers

Business logic, validation, orchestration. Dispatches to SQLite or the Ollama HTTP client as needed.

4

### Ollama REST API (loopback only)

POST `/api/chat` with format: `"json"` for structured, schema-constrained model output.

5

### SQLite (`tauri-plugin-sql`)

Single `aurafit.db` file in the OS app-data directory; WAL mode for concurrent read/write safety.

02 — STACK

# Technology Stack

| Layer             | Technology                  | Version Target         | Rationale                                                                                   |
|-------------------|-----------------------------|------------------------|---------------------------------------------------------------------------------------------|
| App Framework     | Tauri                       | v2.x                   | ~15–30MB RAM overhead vs. Electron's ~200MB+; native OS webview; Rust security model.       |
| UI Framework      | React + TypeScript          | React 18, TS 5.x       | Component reuse, strong typing for the Zod-validated data flowing from Ollama.              |
| Styling           | Tailwind CSS + shadcn/ui    | Tailwind 3.x           | Accessible, consistent, desktop-tuned components without a heavy design-system dependency.  |
| Charts            | Recharts                    | 2.x                    | Declarative, canvas/SVG client-side charts for macro and weight-trend visualization.        |
| Local AI Runtime  | Ollama Server               | latest stable          | Simple local REST endpoint, model management (pull/list), GPU acceleration where available. |
| Vision Model      | llama3.2-vision:11b         | 11B, quantized default | Local multimodal recognition of plated food and portion size.                               |
| Text Model        | llama3.2:3b / qwen2.5:7b    | 3B–7B                  | Meal planning, recipe generation, prompt parsing offline.                                   |
| Database          | SQLite via tauri-plugin-sql | SQLite 3.4x            | Embedded, zero-config, sub-millisecond lookups against pre-loaded USDA data.                |
| Schema Validation | Zod                         | 3.x                    | Runtime validation of every JSON payload returned by Ollama before it touches the database. |

03 — TARGETS

# Non-Functional Requirements

| Category    | Requirement                                         | Target                                                 |
|-------------|-----------------------------------------------------|--------------------------------------------------------|
| Performance | Cold app launch to interactive dashboard            | < 1.5s on a mid-range laptop (SSD, 8GB RAM)            |
| Performance | USDA single-item lookup query                       | < 5ms (sub-millisecond target for indexed exact match) |
| Performance | Vision photo → verification table (GPU-accelerated) | < 12s typical, < 30s CPU-only fallback                 |

|                    |                                        |                                                                                            |
|--------------------|----------------------------------------|--------------------------------------------------------------------------------------------|
| Reliability        | Crash-free session rate                | ≥ 99.5%                                                                                    |
| Reliability        | SQLite write durability                | WAL mode + fsync on commit; no data loss on power failure                                  |
| Security & Privacy | Outbound network calls during core use | Zero — verified via CI network-monitor test                                                |
| Security & Privacy | Data at rest                           | Local file-system permissions only (OS user-scoped); no plaintext transmission ever occurs |
| Portability        | Supported OS targets                   | Windows 10/11 (x64), macOS 12+ (Apple Silicon & Intel), Ubuntu 22.04+/major distros        |
| Resource footprint | Idle RAM (excluding Ollama process)    | < 40MB                                                                                     |
| Accessibility      | Keyboard navigation & contrast         | WCAG 2.1 AA on all core screens                                                            |

## 04 — VISION PIPELINE

## Local Vision AI Pipeline

The vision pipeline is the most technically involved subsystem. It must reliably coerce an inherently unstructured multimodal model response into a strictly typed, database-ready shape.

1

### Capture

User drags a JPEG/PNG onto the drop zone, or captures a frame from the webcam via the browser MediaStream API inside the webview.

2

### Encode

The Rust backend reads the file and encodes it as a base64 string, downscaling to a max 1024px edge to bound inference latency.

3

### Infer

POST `http://localhost:11434/api/chat` with the image, a structured-output prompt, and format: `"json"` targeting a fixed schema.

4

### Validate

The raw JSON string is parsed against a Zod schema on the frontend (and mirrored via `serde` on the Rust side). On failure, one automatic re-prompt is issued with the validation error appended as context.

5

### Verify & Commit

Parsed items populate an editable table. Only on explicit user confirmation is a row written to SQLite.

### 4.1 Request Contract

```
// POST http://localhost:11434/api/chat
{
  "model": "llama3.2-vision:11b",
  "messages": [{
    "role": "user",
    "content": "Identify each food item on this plate. For each item, estimate its weight in grams and its calories, protein, carbs, and fat. Respond only in the given JSON schema.",
    "images": ["<base64-encoded-jpeg>"]
  }],
  "format": "json",
  "stream": false
}
```

## 4.2 Response Schema (Zod)

```
// schemas/mealAnalysis.ts
import { z } from "zod";

export const FoodItemSchema = z.object({
  name: z.string(),
  estimated_grams: z.number().positive(),
  calories: z.number().nonnegative(),
  protein_g: z.number().nonnegative(),
  carbs_g: z.number().nonnegative(),
  fat_g: z.number().nonnegative(),
  confidence: z.enum(["low", "medium", "high"])
});

export const MealAnalysisSchema = z.object({
  items: z.array(FoodItemSchema).min(1),
  total_calories: z.number().nonnegative()
});
```

### DESIGN DECISION

A confidence field is requested per item so the Verification UI can visually flag "low confidence" rows for closer user review, rather than treating every estimate as equally reliable.

05 — DATA & TRUST

# Data Flow & Security

| Concern          | Approach                                                                                                                                                    |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Image data       | Held in memory / temp buffer only for the duration of inference; not persisted unless the user opts to attach a thumbnail to the log entry, stored locally. |
| Network egress   | Tauri CSP restricts the webview to http://localhost:11434 and local asset protocol only; no other origin is reachable.                                      |
| Data at rest     | SQLite file in the OS-standard app-data directory, protected by OS file permissions (no additional app-layer encryption in v1).                             |
| Telemetry        | Off by default. If enabled, crash reports contain stack traces only — never meal content, photos, or biometric values.                                      |
| Data portability | Full CSV/JSON export command reachable from Settings at any time, with no throttling or paywall.                                                            |

06 — RESILIENCE

# Error Handling & Resilience

| Failure Mode              | Detection                             | Recovery                                                                          |
|---------------------------|---------------------------------------|-----------------------------------------------------------------------------------|
| Ollama daemon not running | Connection refused on localhost:11434 | In-app banner with a one-click "Start Ollama" helper and setup-wizard link        |
| Required model not pulled | 404 from /api/chat                    | Guided ollama pull flow with live progress                                        |
| Malformed JSON response   | Zod parse failure                     | One automatic re-prompt; then fallback to manual entry form                       |
| SQLite write failure      | Query error / disk full               | Transaction rollback, non-destructive error toast, entry held in memory for retry |
| Corrupt database file     | Failed integrity check on launch      | Restore from last automatic pre-migration backup; user notified                   |

07 — PLATFORMS

# Cross-Platform Notes

**Windows**

MSI installer, code-signed. Ollama installed as a background service; app detects it via the loopback health check on launch.

**macOS**

DMG, notarized for Gatekeeper. Apple Silicon builds benefit from Metal-accelerated inference in Ollama automatically.

**Linux**

ApplImage and .deb targets. GPU acceleration depends on the user's local CUDA/ROCm setup; CPU fallback always available.